EXPERIMENT - 8

Transactions and Isolation

AIM

To implement banking-style transactions using BEGIN, COMMIT, ROLLBACK and SAVEPOINT, simulate partial transaction failures, and demonstrate isolation levels and concurrency anomalies using two database sessions.

1. CREATE BANKING TABLES

```
CREATE DATABASE IF NOT EXISTS Experiment8;
USE Experiment8;
CREATE TABLE Bank_Account (
 Account_ID INT PRIMARY KEY, Account_Holder VARCHAR(50),
 Balance DECIMAL(12,2) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE Transaction_Log (
 Log_ID INT AUTO_INCREMENT PRIMARY KEY, From_Account INT,
 To_Account INT, Amount DECIMAL(12,2),
 Log_Time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
INSERT INTO Bank_Account VALUES
(101,'Aarav Sharma',50000),(102,'Riya Verma',30000),
(103,'Karan Mehta',20000);
SELECT * FROM Bank_Account ORDER BY Account_ID;
```

OUTPUT: INITIAL BALANCES

Account_ID	Account_Holder	Balance
101	Aarav Sharma	50000.00
102	Riya Verma	30000.00
103	Karan Mehta	20000.00

2. COMMIT A SUCCESSFUL TRANSFER

```
START TRANSACTION;
UPDATE Bank_Account SET Balance=Balance-5000 WHERE Account_ID=101;
UPDATE Bank_Account SET Balance=Balance+5000 WHERE Account_ID=102;
INSERT INTO Transaction_Log (From_Account,To_Account,Amount)
VALUES (101,102,5000);
COMMIT;
SELECT * FROM Bank_Account ORDER BY Account_ID;
```

OUTPUT

Account_ID	Account_Holder	Balance
101	Aarav Sharma	45000.00
102	Riya Verma	35000.00
103	Karan Mehta	20000.00

3. ROLLBACK A FAILED TRANSFER

```
START TRANSACTION;
UPDATE Bank_Account SET Balance=Balance-7000 WHERE Account_ID=101;
UPDATE Bank_Account SET Balance=Balance+7000 WHERE Account_ID=999;
ROLLBACK;
SELECT * FROM Bank_Account ORDER BY Account_ID;
```

OUTPUT: ROLLBACK RESTORES THE COMMITTED BALANCES

Account 101 = 45000.00 | Account 102 = 35000.00 | Account 103 = 20000.00

4. SAVEPOINT FOR PARTIAL FAILURE

```
START TRANSACTION;
UPDATE Bank_Account SET Balance=Balance-3000 WHERE Account_ID=101;
SAVEPOINT after_debit;
UPDATE Bank_Account SET Balance=Balance+3000 WHERE Account_ID=103;
SAVEPOINT after_credit;
UPDATE Bank_Account SET Balance=Balance-1000 WHERE Account_ID=102;
ROLLBACK TO SAVEPOINT after_credit;
COMMIT;
SELECT * FROM Bank_Account ORDER BY Account_ID;
```

OUTPUT

Account_ID	Account_Holder	Balance
101	Aarav Sharma	42000.00
102	Riya Verma	35000.00
103	Karan Mehta	23000.00

The update after after_credit is rolled back; the earlier debit and credit remain committed.

5-6. DIRTY READ - TWO SESSIONS

Session 1: run the update and leave it uncommitted. Session 2: run the read while Session 1 is still open.

```
-- SESSION 1
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
UPDATE Bank_Account SET Balance=Balance+10000 WHERE Account_ID=101;
SELECT Account_ID, Balance FROM Bank_Account WHERE Account_ID=101;
-- Do not commit yet.

-- SESSION 2 (while Session 1 is uncommitted)
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
SELECT Account_ID, Balance FROM Bank_Account WHERE Account_ID=101;
COMMIT;
-- Return to Session 1: ROLLBACK;
```

OUTPUT: SESSION 2 READS THE UNCOMMITTED VALUE

Account_ID	Balance (dirty read)
101	52000.00

7-9. NON-REPEATABLE READ - TWO SESSIONS

```
-- SESSION 1
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT Balance FROM Bank_Account WHERE Account_ID=102;
-- SESSION 2
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
UPDATE Bank_Account SET Balance=Balance+2000 WHERE Account_ID=102;
COMMIT;
-- SESSION 1 reads again, then commits:
SELECT Balance FROM Bank_Account WHERE Account_ID=102;
COMMIT;
```

OUTPUT: SESSION 1 READS THE ROW BEFORE AND AFTER SESSION 2 COMMITS

First read	Second read
35000.00	37000.00

The same row returns a different value on the second read at READ COMMITTED.

10. PHANTOM READ USING TWO SESSIONS

```
-- SESSION 1
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT COUNT(*) AS Accounts_Above_40000
FROM Bank_Account WHERE Balance>40000;
-- SESSION 2
INSERT INTO Bank_Account VALUES (104,'Neha Gupta',60000);
COMMIT;
-- SESSION 1 repeats the count, then commits:
SELECT COUNT(*) AS Accounts_Above_40000
FROM Bank_Account WHERE Balance>40000;
COMMIT;
```

OUTPUT: A NEW ROW APPEARS IN THE SECOND RESULT

Before Session 2 insert	After Session 2 insert
1	2

11. ISOLATION LEVEL COMPARISON

```
SELECT @@transaction_isolation AS Current_Isolation_Level;
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT @@transaction_isolation AS Isolation_Level;
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT @@transaction_isolation AS Isolation_Level;
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT @@transaction_isolation AS Isolation_Level;
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT @@transaction_isolation AS Isolation_Level;
```

Setting	Reported isolation level
Current (MySQL default)	REPEATABLE-READ
READ UNCOMMITTED	READ-UNCOMMITTED
READ COMMITTED	READ-COMMITTED
REPEATABLE READ	REPEATABLE-READ
SERIALIZABLE	SERIALIZABLE

RESULT

COMMIT preserves the successful transfer, ROLLBACK cancels the failed transfer, and SAVEPOINT preserves only the intended partial work. The two-session examples demonstrate dirty, non-repeatable and phantom reads. Isolation-level values may reflect the server configuration.